Automated Data Pipeline for E-commerce Sales Reporting Using Airflow and AWS S3

Soichiro Tanabe 29/03/2026

Table of Contents

1. Executive Summary 2
2. Business Scenario 2
3. Objectives 2
4. Scope 3
5. Architecture Overview 4
6. Technologies Used 4
7. Technology Selection and Justification 5
8. Data Pipeline Workflow (DAG Design) 6
9. AWS S3 Structure 8
10. AWS configuration 9
11. How to Run the Project 10
12. Example Output: 12
13. Key Learnings 12
14. Challenges and Solutions 12
15. Future Improvements 13
16. References 13
17. Secondary Sources 14

1. Executive Summary

This project presents the design and implementation of an automated data pipeline to analyse e-commerce sales data and generate weekly performance reports. The primary objective is to replace manual data processing with a scalable and reproducible workflow, improving efficiency and enabling faster access to business insights.

The solution is built using Apache Airflow for workflow orchestration and Amazon Web Services S3 for cloud-based data storage. The pipeline follows an ETL approach, where data is extracted from S3, transformed using Python libraries such as Pandas and Matplotlib, and output as structured reports in PDF format. These reports include key performance indicators (KPIs), sub-category analysis, and revenue trends.

The system is deployed using Docker to ensure a consistent and reproducible environment. Airflow DAGs are used to manage task dependencies and enable automated scheduling, supporting reliable and repeatable execution of the pipeline.

The project demonstrates practical skills in data engineering, including pipeline design, cloud integration, workflow orchestration, and debugging in distributed environments. Overall, the solution provides a foundation for scalable data processing systems and highlights the benefits of automation in analytical workflows.

2. Business Scenario

Data analysts are required to analyse sales data on a daily and weekly basis, focusing on key performance indicators (KPIs) such as revenue and order volume. However, this process is often manual, requiring significant time to clean, process, and analyse the data.

Automating the generation of weekly reports reduces the time spent on repetitive tasks, enabling analysts to access insights more quickly. This allows them to focus on higher-value analytical activities, such as deeper performance analysis and decision-making. Therefore, an automated data pipeline that produces weekly reports can significantly improve efficiency and support timely business insights.

3. Objectives

The primary objective of this project is to design and implement an automated data pipeline to replace manual data processing workflows. The system aims to ensure data consistency and reproducibility through structured processing and controlled execution. In addition, the project seeks to enable scalable cloud-based storage using Amazon S3 for efficient data management. Finally, it demonstrates the use of Apache Airflow for workflow orchestration, utilising Directed Acyclic Graphs (DAGs) to manage and schedule the end-to-end data processing tasks.

4. Scope

This project utilises an e-commerce sales dataset obtained from Kaggle (Sharmajicoder, n.d.), which is distributed under the MIT License. This license permits free use, modification, and distribution of the data, provided that the original copyright notice is retained.

The scope of the project includes the analysis of sales data to generate automated reports focusing on key business metrics. The analysis covers the following components:

- Weekly summary, including total revenue, gross revenue, average order value (AOV), and week-over-week (WoW) growth
- Bar chart visualisation of sub-category revenue
- Sub-category performance table
- Four-week revenue trend analysis
- Four-week order trend analysis

The project is limited to batch processing of historical data and does not include real-time data streaming or interactive dashboards.

In addition, the project incorporates Apache Airflow for workflow orchestration and Amazon Web Services (AWS), specifically Amazon S3, for cloud-based data storage. These technologies support the automation and scalability of the pipeline. However, the scope does not include advanced cloud services such as data warehousing, real-time processing, or user-facing applications.

5. Architecture Overview

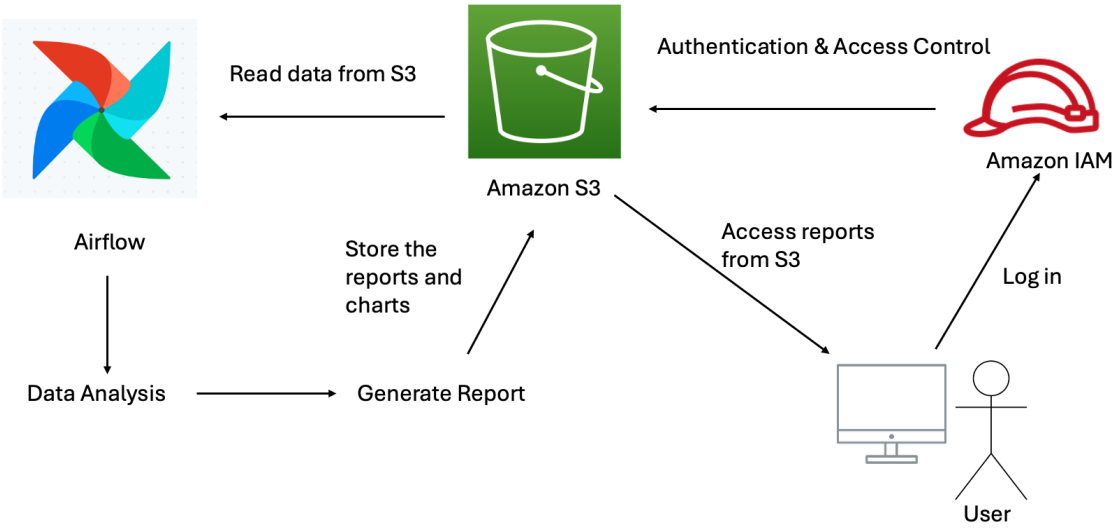


Figure 1: Architecture of the Airflow-Based Data Pipeline with AWS S3 and IAM

The system is designed as an automated ETL pipeline orchestrated by Apache Airflow. Airflow retrieves data from Amazon S3 and performs data analysis, including transformation and aggregation tasks, to generate reports and visualisations. These outputs, such as PDF reports and charts, are then stored back in Amazon S3.

Amazon S3 serves as the central storage layer, holding both input data and processed outputs. Users access the generated reports directly from S3. Access to the system is controlled through AWS Identity and Access Management (IAM), which ensures secure authentication and authorisation before users can retrieve reports.

This architecture enables a scalable, secure, and automated workflow for data processing and reporting.

6. Technologies Used

The following technologies were used to design and implement the data pipeline:

- **Python (Pandas, Matplotlib, ReportLab):** Used for data processing, analysis, visualisation, and report generation
- **Apache Airflow:** Used to orchestrate and schedule the ETL workflow using DAGs
- **Amazon S3:** Used as cloud storage for raw data and generated outputs
- **Docker:** Used to create a consistent and reproducible environment for running Airflow

7. Technology Selection and Justification

7.1 Python

Python was used as the primary programming language due to its ability to support data processing, API interaction, and integration with cloud services. Libraries such as *requests* and Boto3 enable efficient data extraction and communication with AWS services, while tools such as Pandas, Matplotlib, and ReportLab support data analysis, visualisation, and report generation. This makes Python suitable for building end-to-end data pipelines (Lalić, 2024).

7.2 AWS

AWS was chosen as AWS is widely recognised for its flexibility, extensive service offerings, and global scale, making it a preferred choice for large-scale data pipelines (Saim, 2026)

Amazon S3 (Simple Storage Service) was selected as the primary storage solution due to its scalability and durability, making it suitable for cloud-based data pipelines (Segni, 2025). In addition, S3 can function as a central storage layer for analytics and data processing workflows, supporting efficient data organisation and accessibility (Balchandani, 2025).

The Boto3 library, which serves as the official Python SDK for AWS, enables programmatic interaction with services such as Amazon S3. This allows data to be uploaded and managed within the pipeline, facilitating integration between Python-based workflows, including those orchestrated by Apache Airflow, and AWS services (Lalić, 2024b).

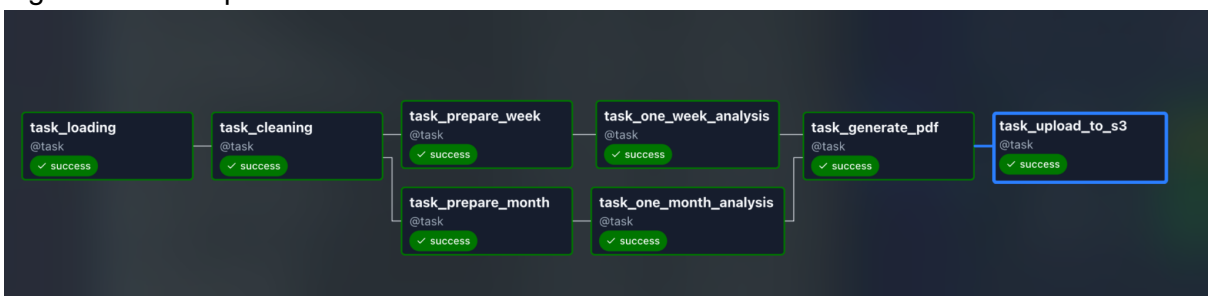
7.3 Airflow & Docker

Apache Airflow was deployed using Docker and Docker Compose to provide a consistent and reproducible environment. This setup enabled efficient dependency management, isolation from the host system, and portability across environments, which are key advantages of containerisation technologies (Docker, 2026; Susnjara and Smalley, 2026).

Airflow was used as the orchestration tool to manage and schedule the data pipeline through Directed Acyclic Graphs (DAGs), enabling automated execution of tasks and clear definition of task dependencies.

8. Data Pipeline Workflow (DAG Design)

Figure 2: Data Pipeline Workflow



8.1 Data Loading (Ingestion)

The pipeline begins by loading raw data from Amazon S3 using Airflow's S3Hook. The dataset (`amazon_sales_dataset.csv`) is retrieved from the *raw* layer and converted into a pandas DataFrame for further processing. The data is then temporarily stored within the Airflow environment to enable downstream tasks.

8.2 Data Cleaning

The cleaning step ensures data quality and consistency. Key operations include parsing date fields, converting numeric columns, and handling invalid or missing values using safe type conversion methods (`errors='coerce'`). Additionally, a derived metric (*gross_sales*) is calculated while handling potential division errors.

8.3 Data Preparation (Filtering & Structuring)

The cleaned dataset is filtered into two analytical subsets:

- a **one-week dataset** for short-term analysis
- a **one-month dataset** for trend analysis

This is achieved by filtering based on the `delivery_date` column and generating time-based labels (e.g., week identifiers) for aggregation.

8.4 One-Week Analysis (KPI Calculation)

The one-week dataset is used to calculate key performance indicators (KPIs), including:

- total revenue
- gross revenue

- total orders
- average order value (AOV)

Additionally, data is aggregated by product sub-category using group-by operations to support detailed performance analysis.

8.5 One-Month Analysis (Trend Analysis)

The one-month dataset is used to generate weekly summaries, including total revenue and order counts. Week-over-week (WoW) growth is calculated to identify performance trends over time. These aggregated datasets form the basis for trend visualisation in the report.

8.6 Report Generation (PDF & Visualisation)

A PDF report is generated using the ReportLab library. The report includes:

- KPI summary tables
- sub-category performance charts
- revenue and order trends

Visualisations are created using Matplotlib and embedded into the PDF, producing an executive-style report.

8.7 Data Storage (Upload to S3)

Finally, the generated outputs (PDF report and charts) are uploaded back to Amazon S3 using Airflow's S3Hook. Files are organised into structured folders (e.g., `reports/pdf/`, `reports/revenue_chart/`), ensuring clear separation of outputs and easy accessibility.

8.8 Assumptions and Design Considerations

Due to the limited size and time range of the dataset, a demonstration mode was implemented to simulate a fixed execution date (Figure: Demo Mode). This allows consistent testing and validation of the pipeline logic, particularly for time-based transformations such as weekly and monthly aggregations. In a production setting, this configuration can be disabled to enable real-time automated scheduling.

Figure 3: Demo Mode

```
# ------------------------------------------------
# DEMO MODE CONFIGURATION
# ------------------------------------------------
DEMO_MODE = True  #  Change to False for real weekly automation
FORCED_DATE = datetime(2026, 2, 7)
```

9. AWS S3 Structure

Figure 4: S3 Structure

```
s3://soichiro-airflow-output-2026/

├── raw/
│   └── (raw data files)

└── reports/
    ├── pdf/
    │   └── Weekly_Performance_Report.pdf
    ├── orders_chart/
    │   └── (orders chart images)
    ├── revenue_chart/
    │   └── (revenue chart images)
    └── subcategory_chart/
        └── (subcategory chart images)
```

The S3 bucket was structured into logical layers, including a *raw* folder for ingested data and a *reports* folder containing generated outputs such as PDF reports and visualisations(Figure: reports folder). This structure supports clear data organisation and separation of processing stages.

Figure 5: reports folder

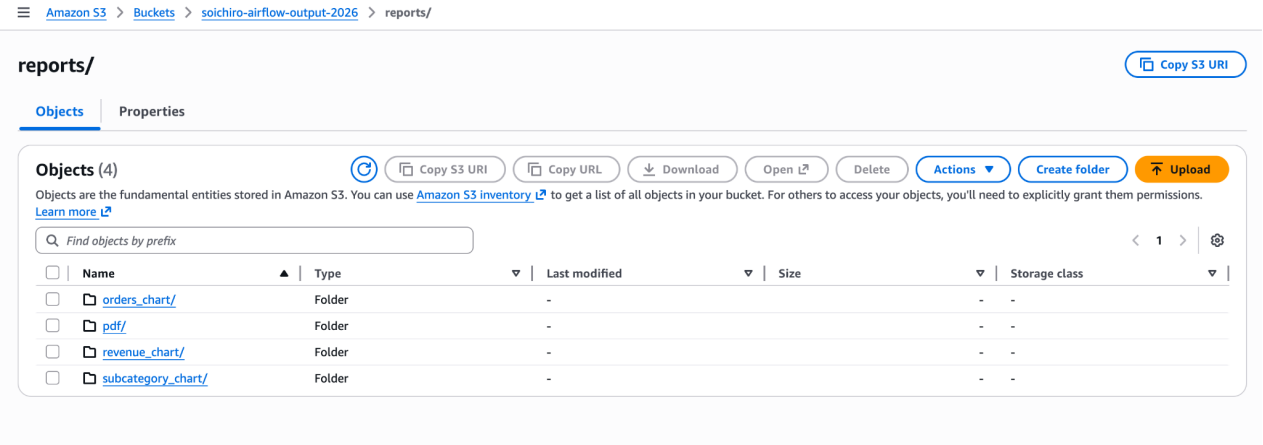
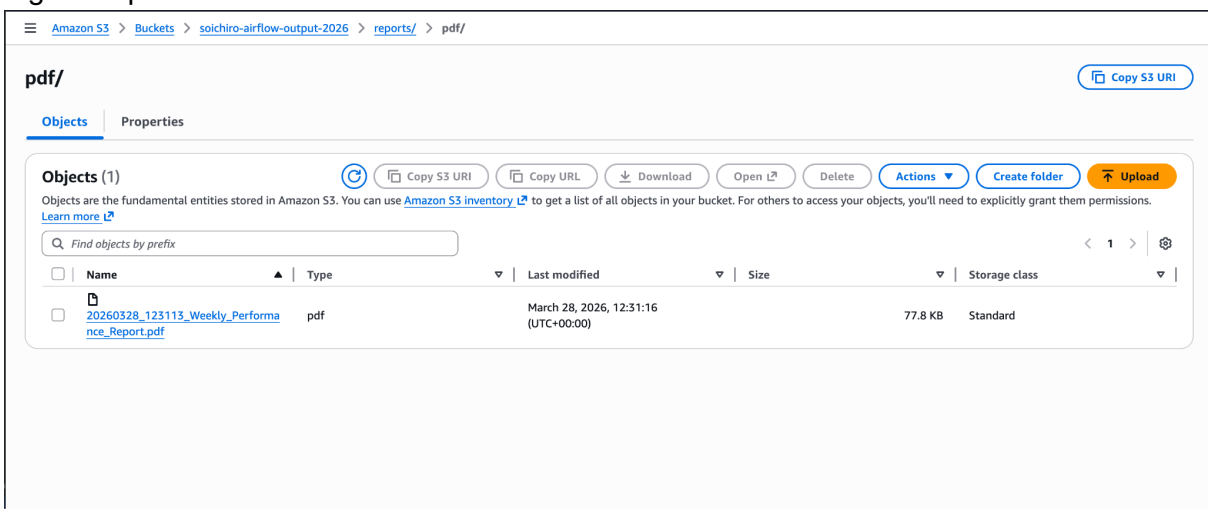


Figure 6: pdf folder



Inside the PDF folder, the report is stored.

10. AWS configuration

An IAM user (*airflow-user*) was created to manage access to AWS resources, rather than using the root account. The user was assigned appropriate permissions, including read and write access to Amazon S3, to support secure data operations within the pipeline. This approach follows established security best practices, which recommend avoiding the use of the root account for routine activities and instead applying the principle of least privilege through IAM users and roles (Amazon Web Services, 2026). Furthermore, the root account possesses unrestricted access privileges and therefore should be reserved only for critical administrative tasks (Haber, 2022).

For security reasons, hardcoding credentials in Python files should be avoided, as it can expose sensitive information (Aikido, 2025). In Apache Airflow, connections are the

recommended method for storing credentials, as sensitive data such as passwords can be securely managed (Fishbein, 2021). Therefore, access credentials were configured using environment variables and integrated into Airflow through connection settings, ensuring secure and seamless communication between the pipeline and AWS services.

11. How to Run the Project

The data pipeline is orchestrated using Apache Airflow and deployed using Docker. The following steps outline how the system can be executed:

11.1 Starting Airflow

Airflow services are started using Docker Compose:

```
docker compose up
```

This command initialises all required components, including the scheduler, web server, and metadata database.

11.2 Accessing the Airflow Interface

Once running, the Airflow web interface can be accessed via:

```
http://localhost:8080
```

The interface allows monitoring of workflows (DAGs), task execution, and logs.

11.3 Triggering the DAG

The pipeline can be executed manually using the Airflow CLI:

```
airflow dags trigger amazon_sales_report_etl_AWS
```

Alternatively, the DAG can be triggered directly from the Airflow web interface.

11.4 Automation

The DAG is configured with a daily schedule, enabling automated execution of the data pipeline and generation of reports without manual intervention.

11.5 Execution Evidence

Evidence of successful execution is demonstrated through Airflow screenshots (e.g., Graph View and Tree View), showing task dependencies and successful task completion.

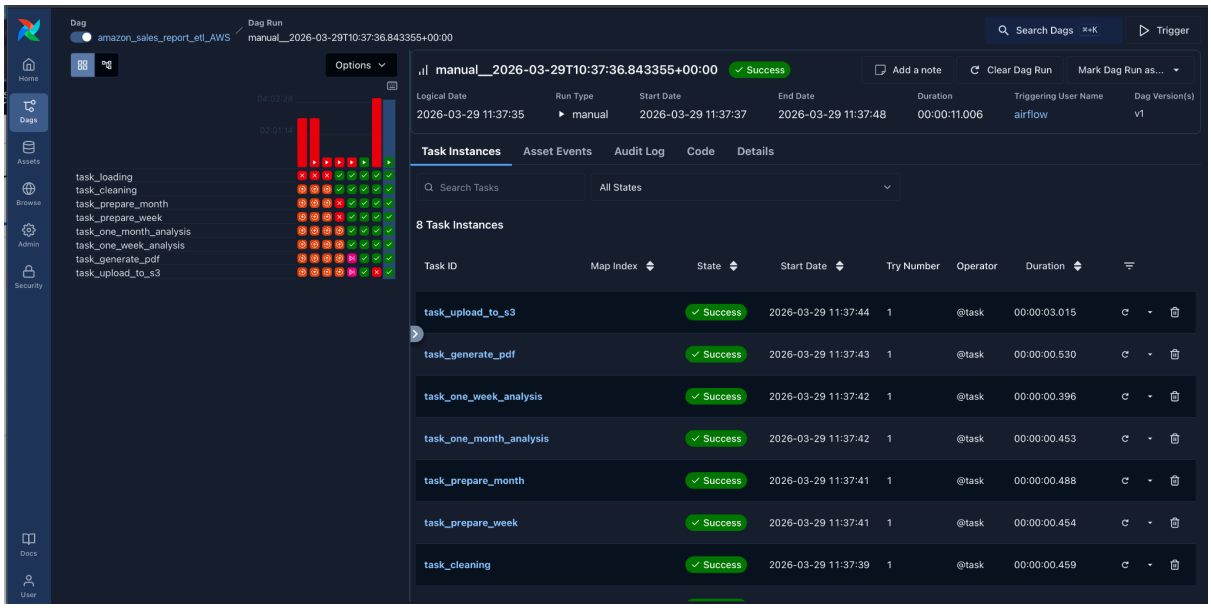


Figure 7: Dag

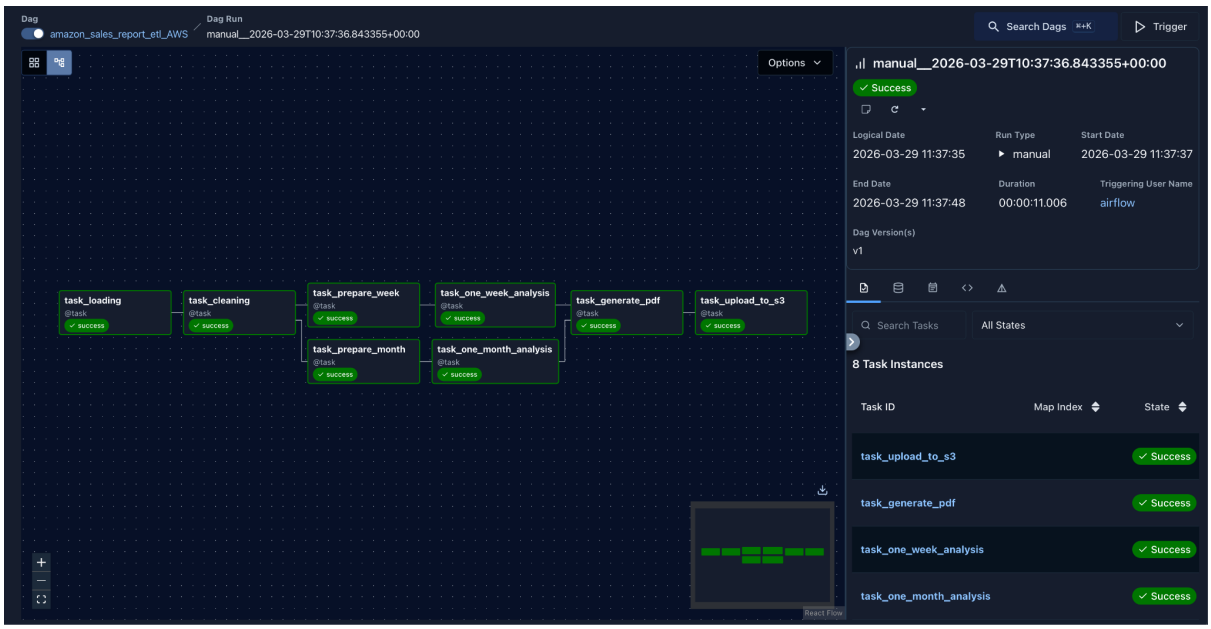


Figure 8: Graph View

12. Example Output:

An example of the generated report output is available in the GitHub repository for reference.

13. Key Learnings

- Developed practical experience in designing Airflow DAGs, including task dependencies and workflow orchestration
- Learned the importance of modular pipeline design to improve maintainability and scalability
- Gained hands-on experience with AWS services, particularly S3 for storage and IAM for secure access control
- Improved debugging skills when working with distributed systems, including resolving issues between containers and the Airflow scheduler
- Understood the differences between local and cloud-based execution, including environment configuration and deployment challenges

14. Challenges and Solutions

14.1 Path inconsistencies

Initially, the pipeline was developed and tested using local file paths to ensure that Airflow execution worked correctly. However, when integrating Amazon S3 for cloud storage, the file paths and data handling logic needed to be adapted to support S3-based storage instead of local directories. This introduced inconsistencies between environments. The issue was resolved by updating the code to use S3-compatible paths and standardising how file locations were managed across all tasks.

14.2 File visibility issues in container environment

During development, output files were not immediately visible after execution due to differences between the container environment and the local file system. Files generated by Airflow tasks were stored within the container rather than the expected local directory. This issue was resolved by identifying the correct execution path (`/opt/airflow/output/`) and ensuring that all tasks consistently wrote output files to this location.

14.5 S3 permission errors

Initial attempts to upload data to Amazon S3 resulted in permission errors due to insufficient IAM policies. This was resolved by correctly configuring IAM roles and attaching the required permissions for S3 access.

15. Future Improvements

15.1 Integration with data warehouses

Data warehouses are designed to support analytical querying and business intelligence by integrating and structuring large volumes of data (Ralph Kimball and Margy Ross, 2013). This pipeline could be extended to load processed data into a data warehouse such as Amazon Web Services Redshift or Google Cloud BigQuery to support scalable analytics and querying (Amazon Web Services, n.d.; Google Cloud, n.d.).

15.2 Monitoring and alerting

Basic monitoring was achieved using Airflow logs and task status tracking. This could be further enhanced by implementing automated alerting mechanisms, such as email notifications or failure alerts, to improve pipeline reliability and responsiveness.

16. References

Aikido (2025) *Prevent hardcoded secrets in Python*. Available at: https://www.aikido.dev/code-quality/rules/prevent-hardcoded-secrets-in-python (Accessed: 28 March 2026).

Amazon Web Services (n.d.) *What is Amazon Redshift?* Available at: https://aws.amazon.com/redshift/ (Accessed: 29 March 2026).

Amazon Web Services (2026) *Security best practices in IAM*. Available at: https://docs.aws.amazon.com/IAM/latest/UserGuide/best-practices.html (Accessed: 28 March 2026).

Docker (2026) *Docker overview*. Available at: https://docs.docker.com/get-started/docker-overview/ (Accessed: 28 March 2026).

Fishbein, N. (2021) *Misconfigured Airflows leak thousands of credentials from popular services*. Available at: https://intezer.com/blog/misconfigured-airflows-leak-credentials/ (Accessed: 28 March 2026).

Google Cloud (n.d.) *BigQuery*. Available at: https://cloud.google.com/bigquery (Accessed: 29 March 2026).

Haber, M.J. (2022) *AWS root vs IAM user: What to know & when to use them*. Available at: https://www.beyondtrust.com/blog/entry/aws-root-vs-iam-user-what-to-know-when-to-use-them (Accessed: 28 March 2026).

Kimball, R. and Ross, M. (2013) *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling*. 3rd edn. Wiley.

Lalić, M. (2024b) *Complete data pipeline with Airflow and AWS*. Available at: https://www.linkedin.com/pulse/complete-data-pipeline-airflow-aws-milo%C5%A1-lali%C4%87-f92nf/ (Accessed: 28 March 2026).

Lalić, M. (2024a) *Complete data pipeline with Airflow and AWS*. Available at: https://www.linkedin.com/pulse/complete-data-pipeline-airflow-aws-milo%C5%A1-lali%C4%87-f92nf/ (Accessed: 28 March 2026).

Saim, M. (2026) *Azure vs AWS: A Complete Comparison of Services, Pricing, and Performance*. Available at: https://costimizer.ai/blogs/azure-vs-aws (Accessed: 28 March 2026).

Segni, S. (2025) *Everything you need to know about AWS S3 for cloud storage*. Available at: https://dev.to/souhailsegni/everything-you-need-to-know-about-aws-s3-for-cloud-storage-3032 (Accessed: 28 March 2026).

Sharmajicoder (n.d.) *E-commerce Sales Dataset*. Available at: https://www.kaggle.com/datasets/sharmajicoder/e-commerce-sales-dataset (Accessed: 29 March 2026).

Susnjara, S. and Smalley, I. (2026) *What is Docker?*. Available at: https://www.ibm.com/topics/docker (Accessed: 28 March 2026).

17. Secondary Sources

AWS Developers(n.d.) *Understanding why Amazon S3 is the best place to build your data lake*. Available at: https://www.youtube.com/watch?v=_abfv7Efr5Q (Accessed: 28 March 2026).

Data Slinger (2024) *Creating a Minimal Airflow 3.0 Docker Compose Setup*. Available at: https://www.youtube.com/watch?v=PbSIVDou17Q (Accessed: 29 March 2026).